

Universidad Nacional de Colombia
Estructuras de Datos — 2026-2

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Cuatro listas enlazadas, MyStack y MyQueue sobre arreglo dinámico circular, medidas con entradas aleatorias de tamaño 10 a 10^7

Estudiante	Raúl Santiago Bermúdez Camacho
Profesor	David Herrera
Monitora	Ángela Camila Siabato Londoño
Tarea	Tarea individual
Repositorio GitHub	https://github.com/USUARIO/lab-estructuras-datos

1. Objetivo

Implementar las estructuras de datos Stack (pila) y Queue (cola) en Java, utilizando tanto arreglos dinámicos como listas enlazadas, realizando un análisis de complejidad de los métodos asociados.

Para ello se implementa la estructura List con listas enlazadas (sencillamente enlazadas sin y con cola, y doblemente enlazadas sin y con cola), y las interfaces MyStack<T> y MyQueue<T> con un arreglo dinámico circular. Se mide el tiempo de ejecución de cada método con entradas aleatorias de tamaño 10, 100, 10^4 , 10^6 y 10^7 , y se compara con la complejidad teórica en notación Big-O.

2. Explicación de la implementación

2.1 Arquitectura del proyecto

El código está en el paquete edu.unal.ed, dividido por responsabilidad. Las estructuras no dependen del código de medición, y la medición no grafica: solo escribe CSV.

Paquete / archivo	Contenido
list	Interfaces MyList<T> y ListNode<T> ; nodos SinglyNode y DoublyNode; las 4 listas.
circular	CircularArray<T> : arreglo dinámico circular propio (crecimiento por duplicación).
stack / queue	Interfaces MyStack<T> y MyQueue<T> ; ArrayStack y ArrayQueue, ambas sobre CircularArray.
benchmark	Medición con System.nanoTime(), estadísticas y escritura de CSV.
Main.java, Operation.java	Orquestan las mediciones. Adaptados del Main.java guía del profesor (se conserva exec(size, method, operation)).
results/graficar.py	Lee los CSV y genera las gráficas, fuera de Java.

2.2 Decisiones de diseño no especificadas por la guía

Decisión	Opción elegida	Motivo
Retorno de Find	La referencia al nodo, como ListNode<T> (null si no está). Erase, AddBefore y AddAfter reciben esa referencia.	La guía pide “retornar la referencia del elemento”. Así se observa la diferencia entre $O(1)$ y $O(n)$ según la lista tenga o no puntero prev.
Lista vacía	Pop/Top lanzan NoSuchElementException.	Error explícito en vez de devolver null ambiguo.
Genéricos	MyList<T>, igual que MyStack<T> y MyQueue<T>.	Coherencia entre las tres estructuras.
delete(n) en la pila	Busca desde la cima hacia la base.	Es el orden natural en que se recorre una pila.
delete(n) en la cola	Busca desde el frente hacia el final.	Orden natural FIFO.
Crecimiento del arreglo	Capacidad inicial 16; se duplica al llenarse. No se reduce.	Duplicar da $O(1)$ amortizado (sección 2.4). Reducir no se pide.
Tamaño máximo de entrada	10^7 (tamaños 10, 100, 10^4 , 10^6 y 10^7).	La guía da 10^8 como ejemplo (“ej.”). Una lista doble de 10^8 Integer ocupa unos 4.4 GB, demasiado para medir con holgura en el equipo de 8 GB; con 10^7 (≈ 0.5 GB) las diferencias $O(1)/O(n)$ ya se ven con claridad.

2.3 Las cuatro listas enlazadas

Todas implementan **MyList<T>** con pushFront, pushBack, popFront, popBack, find, erase, addBefore, addAfter, isEmpty (Empty), size, topFront y topBack, y guardan un contador size para que Empty y Size sean $O(1)$. Las variantes “con cola” mantienen además un puntero tail, y las dobles un puntero prev en cada nodo.

Método	Simple sin cola	Simple con cola	Doble sin cola	Doble con cola
PushFront	O(1)	O(1)	O(1)	O(1)
PushBack	O(n)	O(1)	O(n)	O(1)
PopFront	O(1)	O(1)	O(1)	O(1)
PopBack	O(n)	O(n)	O(n)	O(1)
Find	O(n)	O(n)	O(n)	O(n)
Erase	O(n)	O(n)	O(1)	O(1)
AddBefore	O(n)	O(n)	O(1)	O(1)
AddAfter	O(1)	O(1)	O(1)	O(1)
TopFront	O(1)	O(1)	O(1)	O(1)
TopBack	O(n)	O(1)	O(n)	O(1)
Empty	O(1)	O(1)	O(1)	O(1)
Size	O(1)	O(1)	O(1)	O(1)

Tabla 1. Complejidad teórica de cada método (verde: O(1); ámbar: O(n)).

- **PushBack / TopBack sin cola: O(n)**, porque hay que recorrer desde head hasta el último nodo. Con tail son O(1).
- **PopBack: O(n) salvo en la doble con cola.** Aun conociendo tail, una lista simple no puede llegar al penúltimo nodo sin recorrer desde head. Solo tail.prev lo hace O(1).
- **Erase y AddBefore: O(n) en las simples, O(1) en las dobles.** Aunque se reciba el nodo, en una lista simple hay que buscar su predecesor desde head para re-enlazar. Con prev el predecesor está a un paso.
- **AddAfter: O(1) en todas**, porque insertar después de un nodo conocido solo modifica node.next (y tail si el nodo era el último).
- **Find: O(n) en todas**, porque ninguna lista tiene índice auxiliar.

2.4 Arreglo dinámico circular, MyStack y MyQueue

CircularArray guarda los elementos en un Object[] con dos enteros, head (posición física del primer elemento) y size. El elemento lógico i está en la posición física (head + i) mód capacidad, así que retirar del frente solo avanza head y nunca desplaza elementos. ArrayQueue usa addLast/removeFirst/peekFirst y ArrayStack usa addLast/removeLast/peekLast; la cima de la pila es el último elemento lógico.

Estrategia de crecimiento: duplicar. Cuando size = capacidad, grow() crea un arreglo del doble y copia los elementos en orden lógico (el frente queda en la posición 0). El análisis es el siguiente:

- **Peor caso de una inserción: O(n).** La inserción que encuentra el arreglo lleno copia los n elementos.
- **Costo amortizado: O(1).** En N inserciones desde el arreglo vacío, las copias ocurren al llenar capacidades 16, 32, 64, ..., y suman $16 + 32 + \dots + N/2 + N < 2N$. El costo total es a lo sumo N escrituras + 2N copias = O(N), es decir, O(1) por inserción en promedio.
- **Espacio:** la capacidad nunca pasa de $2 \cdot \text{size} + 16$, porque se duplica solo cuando está lleno.

Por qué circular. En la cola es indispensable: sin el buffer circular, cada dequeue obligaría a desplazar todos los elementos (O(n)) o dejaría espacio perdido al inicio. En la pila no es necesario, porque push y pop ocurren en el mismo extremo y head nunca se mueve. Se usa el mismo buffer para que ambas estructuras compartan la estrategia de crecimiento.

delete(n): O(n). Hace una búsqueda lineal y luego desplaza una posición los elementos que siguen al eliminado, para cerrar el hueco.

MyStack	Big-O	MyQueue	Big-O
push	O(1) amortizado (O(n) peor caso)	enqueue	O(1) amortizado (O(n) peor caso)
pop	O(1)	dequeue	O(1)
peek	O(1)	front	O(1)
isEmpty	O(1)	isEmpty	O(1)
size	O(1)	size	O(1)
delete(n)	O(n)	delete(n)	O(n)

Tabla 2. Complejidad teórica de MyStack y MyQueue.

2.5 Metodología de medición

Unidad: nanosegundos con System.nanoTime(). El Main.java guía usa Instant/Duration en milisegundos, pero una operación O(1) tarda del orden de decenas de nanosegundos: en milisegundos casi todas las mediciones darían 0. nanoTime() es un reloj monotónico que reporta nanosegundos; en el equipo de medición su resolución práctica (la menor diferencia entre dos lecturas seguidas) es de 100 ns. Las gráficas usan microsegundos, como sugiere la guía, para que los números sean legibles.

Qué se mide. Para cada tamaño n se construye la estructura una sola vez, sin cronometrar, con una permutación aleatoria de $0..n-1$ (semilla fija 2026, reproducible). En cada repetición:

- sin cronometrar, se elige al azar un valor de la estructura (así la posición del nodo buscado o borrado es uniforme) y, si el método lo necesita, se busca su nodo;
- se cronometra **una sola llamada** al método;
- sin cronometrar, se deshace su efecto (p. ej., tras popBack se hace pushBack del mismo valor), de modo que la estructura siempre tiene n elementos.

Así el número medido es “cuánto cuesta una operación cuando la estructura tiene n elementos”, que es lo que describe Big-O y lo que muestra la tabla de ejemplo de la guía. Acumular n llamadas bajo un mismo cronómetro, como hace el borrador del profesor, mide otra cosa: para un método O(n) costaría O(n^2) y sería inviable en 10^7 . Ese patrón (el método exec) se conserva para medir el costo amortizado de push/enqueue al construir la pila o la cola con n inserciones (sección 3.3).

n	10^1	10^2	10^4	10^6	10^7
Repeticiones	2000	2000	500	50	20

Tabla 3. Repeticiones por tamaño. Disminuyen con n porque deshacer (y a veces buscar el nodo) cuesta O(n) sin cronometrar.

- **Calentamiento de la JVM:** antes de medir se ejecutan dos pasadas completas hasta $n = 10^4$, cuyos resultados se descartan, para que el compilador JIT optimice el código.
- **Medida reportada: la media recortada al 10 %:** se ordenan las muestras, se descarta el 10 % más bajo y el 10 % más alto (al menos una por extremo) y se promedia el resto. Es un promedio, pero no se deja arrastrar por una pausa aislada del recolector de basura. El CSV incluye además media, mediana, mínimo, máximo y desviación estándar.
- **Autoboxing fuera del cronómetro:** los Integer se crean antes de medir.
- **Separación medición/graficación:** Java solo escribe results/tiempos.csv. results/graficar.py lee el CSV y genera las gráficas en otro proceso, después.
- **Equipo de medición:** Intel Core Ultra 5 125H, 8 GB de RAM, Windows 11 10.0 (amd64), Java 21.0.12.1, heap máximo de 2.7 GB, recolector Parallel GC.

3. Análisis de complejidad: teórico vs. empírico

3.1 Hipótesis

Según la complejidad teórica (tablas 1 y 2), un método $O(1)$ debe tardar lo mismo sin importar n , y uno $O(n)$ debe tardar en proporción a n : por ejemplo, entre $n = 10^4$ y $n = 10^7$ (n crece $\times 1\,000$) su tiempo debería multiplicarse por $\approx 1\,000$. En escala log-log, los $O(1)$ se ven como una línea horizontal y los $O(n)$ como una recta ascendente. En concreto se espera que:

- **(H1)** PushBack y TopBack sean planos solo en las listas con cola, y PopBack solo en la doble con cola.
- **(H2)** Erase y AddBefore muestren la mayor separación entre simples ($O(n)$) y dobles ($O(1)$).
- **(H3)** PushFront, PopFront, AddAfter, TopFront, Empty y Size sean planos en las cuatro listas, y Find crezca linealmente en todas.
- **(H4)** En Stack y Queue, push/pop/peek/enqueue/dequeue/front/isEmpty/size sean planos y delete(n) lineal; el push que dispara el resize crezca linealmente, pero el costo amortizado de n inserciones se mantenga constante.
- **(H5)** En la comparativa, los pares $O(1)$ tengan tiempos del mismo orden, y Find+Erase vs delete ambos crezcan linealmente.

3.2 Parte 1 — Comparativa entre las cuatro implementaciones de List

Las tablas 4 a 7 dan, para cada implementación, el tiempo por operación en microsegundos (media recortada), con el mismo formato que el ejemplo de la guía. Las columnas en ámbar son los métodos $O(n)$ de esa implementación.

SinglyLinkedListNoTail

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter	TopBack
10^1	0.026	0.037	0.022	0.031	0.036	0.035	0.025	0.042	0.043
10^2	0.034	0.200	0.034	0.210	0.170	0.150	0.182	0.044	0.210
10^4	0.036	15.7	0.033	17.1	15.7	9.7	14.2	0.042	11.6
10^6	0.022	1 897	0.015	2 302	2 729	1 083	1 931	1.1	2 058
10^7	0.062	20 577	0.062	19 744	117 349	9 818	18 699	8.7	23 495

Tabla 4. SinglyLinkedListNoTail: tiempo por operación (μ s).

SinglyLinkedListWithTail

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter	TopBack
10^1	0.030	0.036	0.024	0.040	0.047	0.038	0.044	0.047	0.036
10^2	0.025	0.045	0.039	0.232	0.220	0.172	0.150	0.031	0.028
10^4	0.034	0.031	0.018	12.3	14.0	9.0	13.8	0.036	0.015
10^6	0.022	0.825	0.025	1 811	2 014	924	1 728	0.963	0.020
10^7	0.100	2.4	0.050	26 753	133 802	14 398	21 187	5.0	0.006

Tabla 5. SinglyLinkedListWithTail: tiempo por operación (μ s).

DoublyLinkedListNoTail

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter	TopBack
10^1	0.038	0.040	0.032	0.039	0.049	0.033	0.034	0.033	0.039
10^2	0.030	0.176	0.029	0.200	0.194	0.038	0.035	0.030	0.187
10^4	0.020	12.4	0.035	12.5	17.6	0.036	0.056	0.047	13.5
10^6	0.048	1 817	0.068	1 744	2 203	1.2	1.8	1.4	1 781
10^7	0.056	22 709	0.013	17 002	93 081	2.9	3.0	2.9	21 928

Tabla 6. DoublyLinkedListNoTail: tiempo por operación (μ s).

DoublyLinkedListWithTail

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter	TopBack
10^1	0.040	0.027	0.031	0.031	0.058	0.044	0.049	0.048	0.045
10^2	0.036	0.033	0.032	0.038	0.289	0.038	0.039	0.035	0.032
10^4	0.025	0.036	0.036	0.045	18.3	0.035	0.031	0.036	0.029
10^6	0.055	0.065	0.072	0.060	1 864	1.4	1.7	1.8	0.065
10^7	0.019	0.013	0.037	0.013	138 717	2.1	2.2	2.1	0.025

Tabla 7. DoublyLinkedListWithTail: tiempo por operación (μs).

PushBack y PopBack

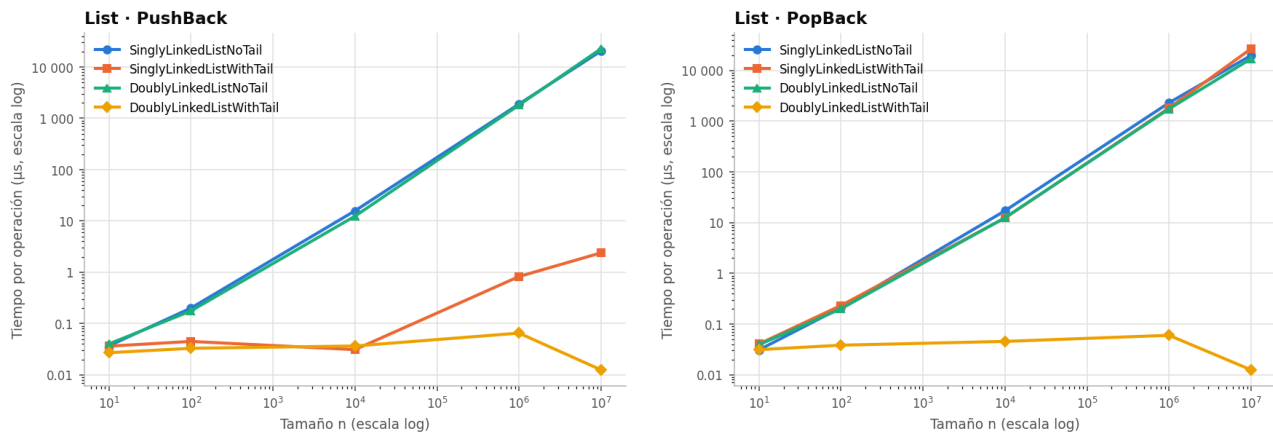


Figura 1. PushBack (izq.) y PopBack (der.) en las 4 listas.

H1 se confirma. Entre $n = 10^4$ y $n = 10^7$ (n crece $\times 1\,000$), PushBack en las listas sin cola crece $\times 1\,307$ (simple) y $\times 1\,835$ (doble), como corresponde a $O(n)$: con $n = 10^7$ tarda $20\,577\,\mu s$ y $22\,709\,\mu s$. Con cola se queda en $2.4\,\mu s$ (simple) y $0.013\,\mu s$ (doble), miles de veces menos. La simple con cola sí sube algo con n grande (de $0.031\,\mu s$ en 10^4), pero mucho menos que n : no es un recorrido sino un efecto de memoria (sección 3.5). En PopBack la simple con cola se comporta como las listas sin cola ($26\,753\,\mu s$ con $n = 10^7$), porque tener tail no le permite llegar al penúltimo nodo, y solo la doble con cola se mantiene constante ($0.013\,\mu s$).

Erase y AddBefore

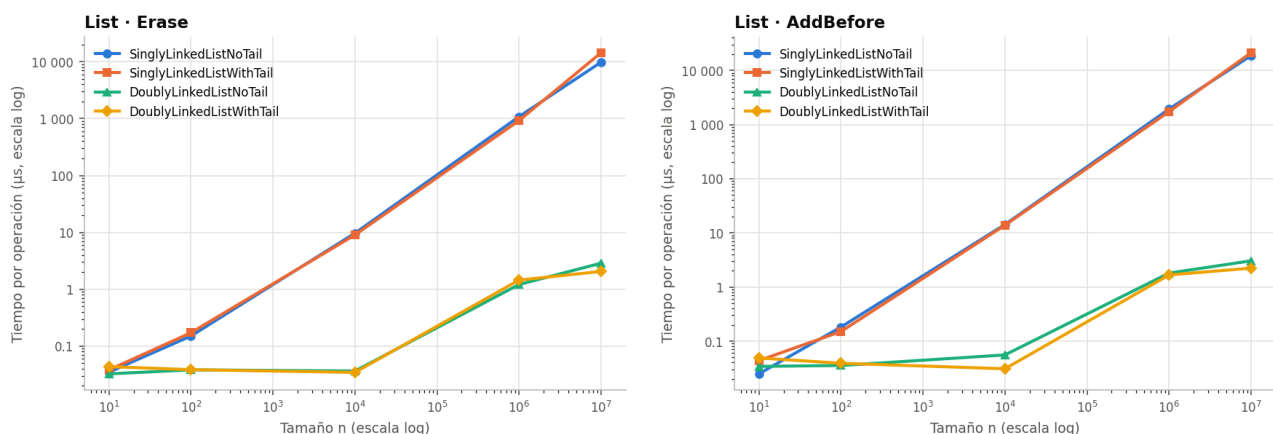


Figura 2. Erase (izq.) y AddBefore (der.) recibiendo el nodo.

H2 se confirma. Con $n = 10^7$, borrar un nodo ya conocido cuesta $9\,818\,\mu s$ en la simple sin cola y $2.1\,\mu s$ en la doble con cola, unas $4\,774$ veces menos. La lista simple recorre desde head para hallar el predecesor (crece $\times 1\,015$ entre 10^4 y 10^7); la doble lo tiene en `node.prev`. AddBefore muestra la

misma separación (21 187 μs en la simple con cola frente a 2.2 μs en la doble con cola).

Métodos $O(1)$ en las cuatro listas y Find

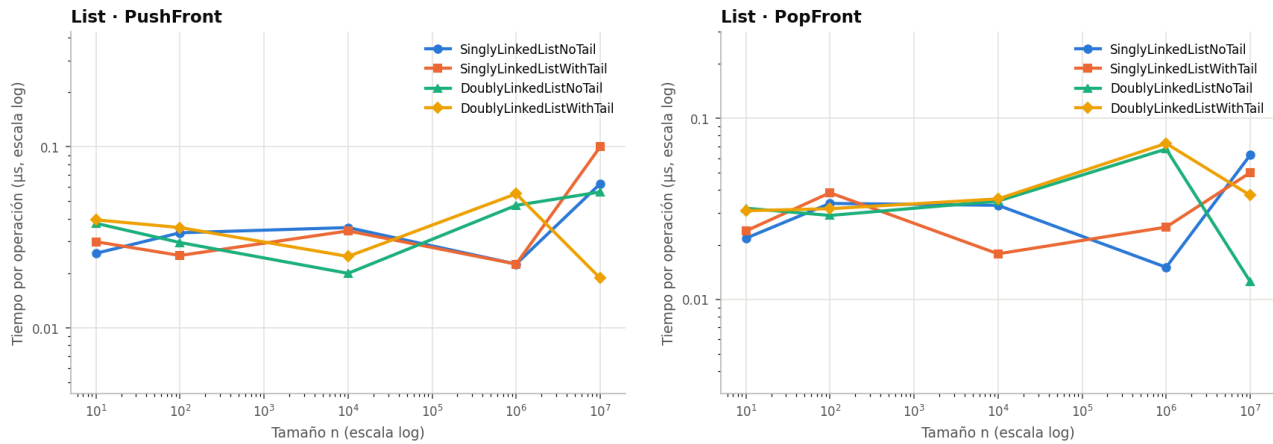


Figura 3. PushFront (izq.) y PopFront (der.): $O(1)$ en las cuatro.

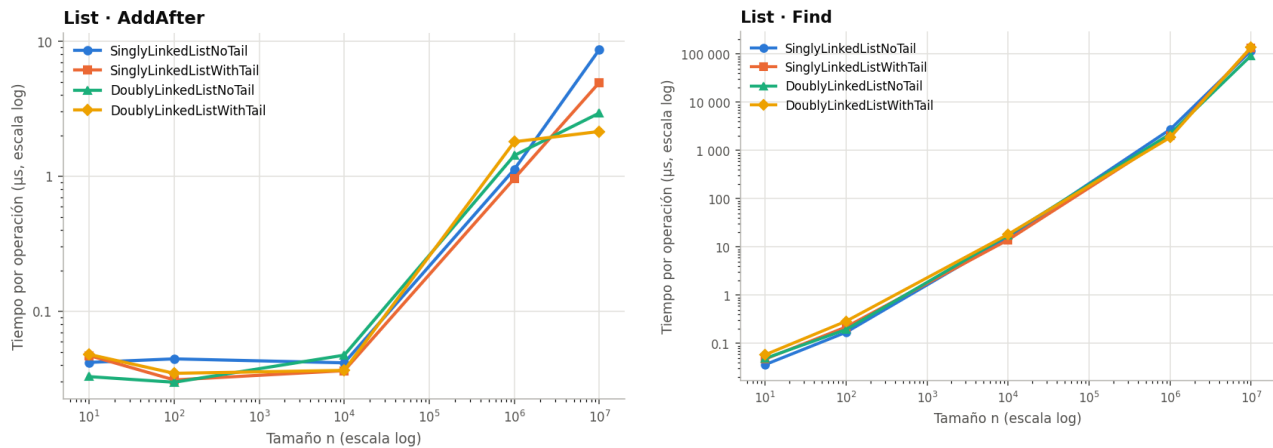


Figura 4. AddAfter (izq.) y Find (der.).

H3 se confirma. PushFront y PopFront no superan 0.100 μs desde $n = 10$ hasta $n = 10^7$ en las cuatro listas: no dependen de n . Find crece $\times 7\,478$ entre 10^4 y 10^7 ($O(n)$) y es casi idéntico en las cuatro, porque todas recorren nodo a nodo. AddAfter es $O(1)$ en todas y no se separa por implementación, a diferencia de AddBefore; su tiempo sube algo con n grande por efectos de memoria (sección 3.5), pero mucho menos que n .

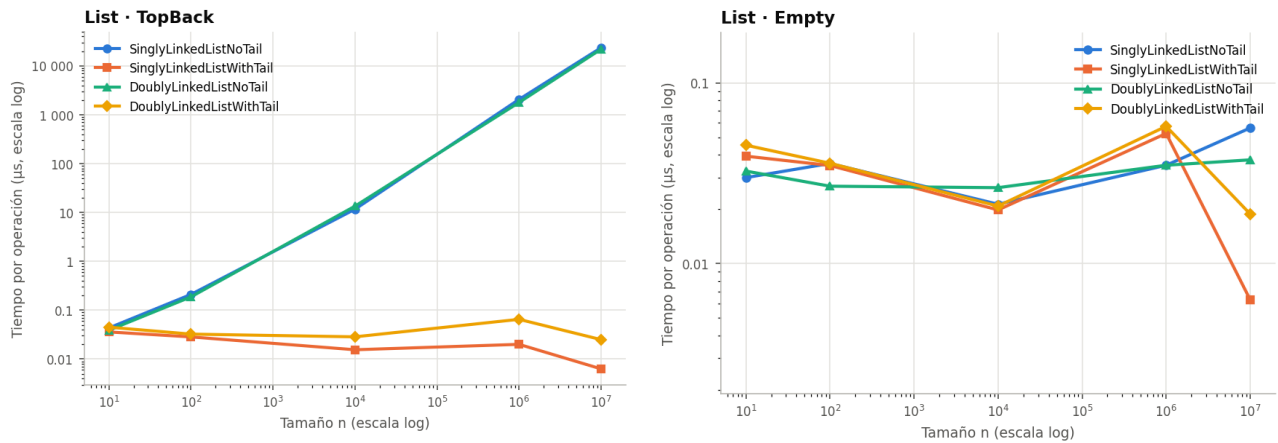


Figura 5. TopBack (izq.) y Empty (der.).

TopBack reproduce el patrón de PushBack: $O(n)$ sin cola y $O(1)$ con cola. Empty, Size y TopFront son constantes en todas porque solo leen un campo.

Las gráficas de TopFront y Size, y las cuatro gráficas por implementación (todos los métodos de una lista en una sola figura, como el ejemplo de la guía), están en results/graficas/ del repositorio.

3.3 Parte 2 — MyStack y MyQueue (arreglo dinámico circular)

n	push	pop	peek	delete	enqueue	dequeue	front	delete
10 ¹	0.110	0.149	0.121	1.1	0.123	0.052	0.066	0.830
10 ²	0.100	0.212	0.142	7.8	0.105	0.212	0.216	7.2
10 ⁴	0.168	0.255	0.139	93.5	0.046	0.050	0.044	47.4
10 ⁶	0.040	0.035	0.048	3 908	0.028	0.052	0.045	4 599
10 ⁷	0.019	0.050	0.037	20 808	0.019	0.044	0.037	19 928

Tabla 8. Tiempo por operación (μs): ArrayStack (columnas 2-5) y ArrayQueue (columnas 6-9). isEmpty y size se comportan como peek.

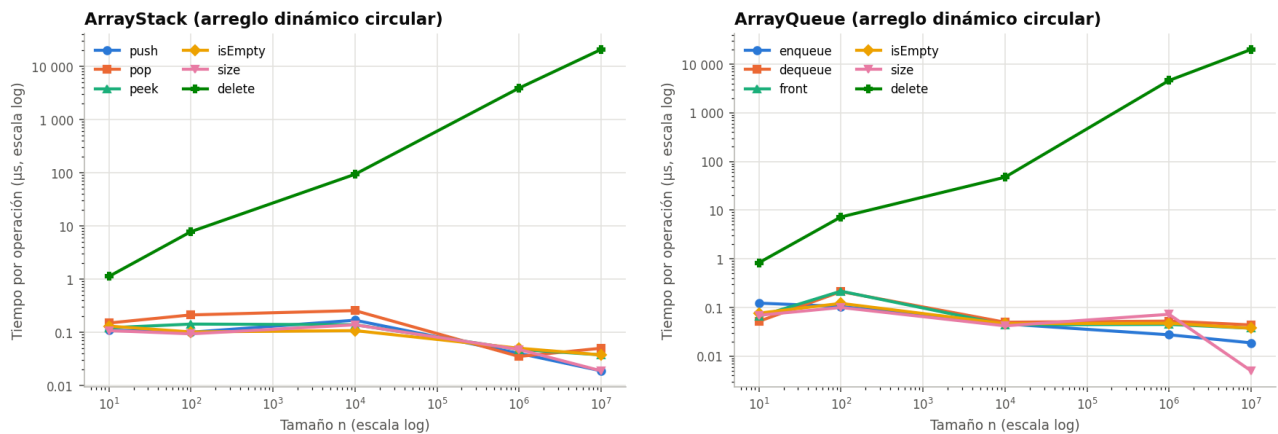


Figura 6. Métodos de ArrayStack (izq.) y ArrayQueue (der.).

H4 se confirma. push, pop, peek, isEmpty y size (pila), y enqueue, dequeue, front, isEmpty y size (cola), no superan 0.255 μs en todo el rango (a lo sumo unos pocos tics del reloj de 100 ns): no dependen de n. delete(n) crece $\times 223$ en la pila y $\times 421$ en la cola entre 10⁴ y 10⁷ (n crece $\times 1\,000$), como corresponde a $O(n)$. En la cola, dequeue sigue siendo $O(1)$ aunque el frente se desplace alrededor del arreglo: la estructura se “rotó” un número aleatorio de posiciones antes de medir, así que los datos dan la vuelta por el final.

Redimensionamiento: peor caso vs. costo amortizado

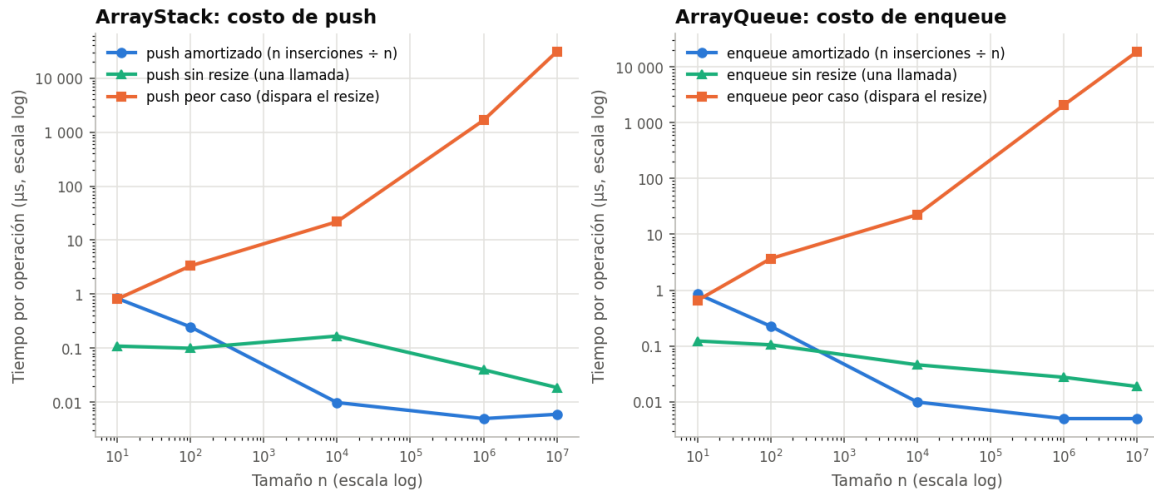


Figura 7. Peor caso (la inserción que dispara la duplicación), una inserción sin resize y costo amortizado (n inserciones desde vacío $\div n$).

La figura 7 separa los tres costos de push/enqueue. La inserción que encuentra el arreglo lleno copia los n elementos y crece linealmente: con $n = 10^7$ tarda 30 687 μs en la pila y 18 743 μs en la cola. Construir la estructura con n inserciones desde vacío, medido con el patrón `exec` del Main del profesor y dividido entre n , cuesta en cambio unos 0.006 μs por inserción, un valor que no crece con n . Esa es la demostración empírica del $O(1)$ amortizado: las copias son cada vez más caras, pero cada vez más raras, y en total suman menos de $2n$. En $n = 10$ el amortizado sale más alto porque el costo fijo de crear el objeto se reparte entre muy pocas inserciones.

3.4 Parte 3 — Métodos equivalentes: List vs. Stack y Queue

Cada método de Stack y Queue se empareja con su equivalente en List, usando para cada par la implementación de List más óptima para ese uso, es decir, la que logra la mejor complejidad posible con el menor costo extra por nodo:

Stack / Queue	Equivalente en List	Lista elegida	Justificación
push, pop, peek, isEmpty, size	PushFront, PopFront, TopFront, Empty, Size	Simple sin cola	La cima de la pila es head: las operaciones son $O(1)$ en las cuatro listas, así que se elige la de menor memoria por nodo (sin tail ni prev), igual que el arreglo, que tampoco guarda enlaces.
enqueue, dequeue, front, isEmpty, size	PushBack, PopFront, TopFront, Empty, Size	Simple con cola	enqueue necesita PushBack en $O(1)$, y eso solo lo logran las variantes con cola. FIFO nunca retira por el final, así que prev no aporta nada: se elige la simple, no la doble.
delete(n) de la pila	Find + Erase	Doble sin cola	delete = buscar + quitar. Con la API find/erase, una lista simple recorre dos veces (Find y luego el predecesor en Erase); en una doble Erase es $O(1)$ y el total es un solo recorrido, igual que el arreglo (búsqueda + desplazamiento). La pila no usa tail, así que basta la doble sin cola.
delete(n) de la cola	Find + Erase	Doble con cola	Mismo argumento, pero la lista que modela la cola necesita tail para enqueue $O(1)$: debe ser doble (Erase $O(1)$) y con cola.

Tabla 9. Pares comparados y lista elegida para cada uno.

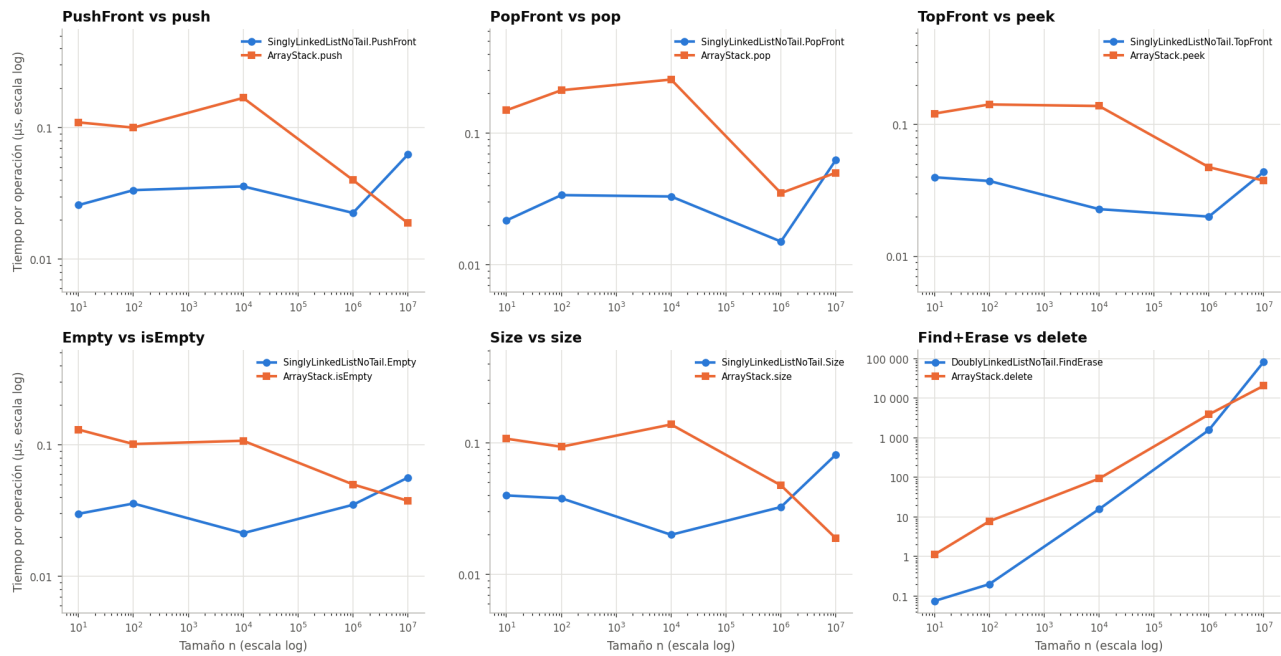


Figura 8. List (simple sin cola; doble sin cola para Find+Erase) vs. ArrayStack.

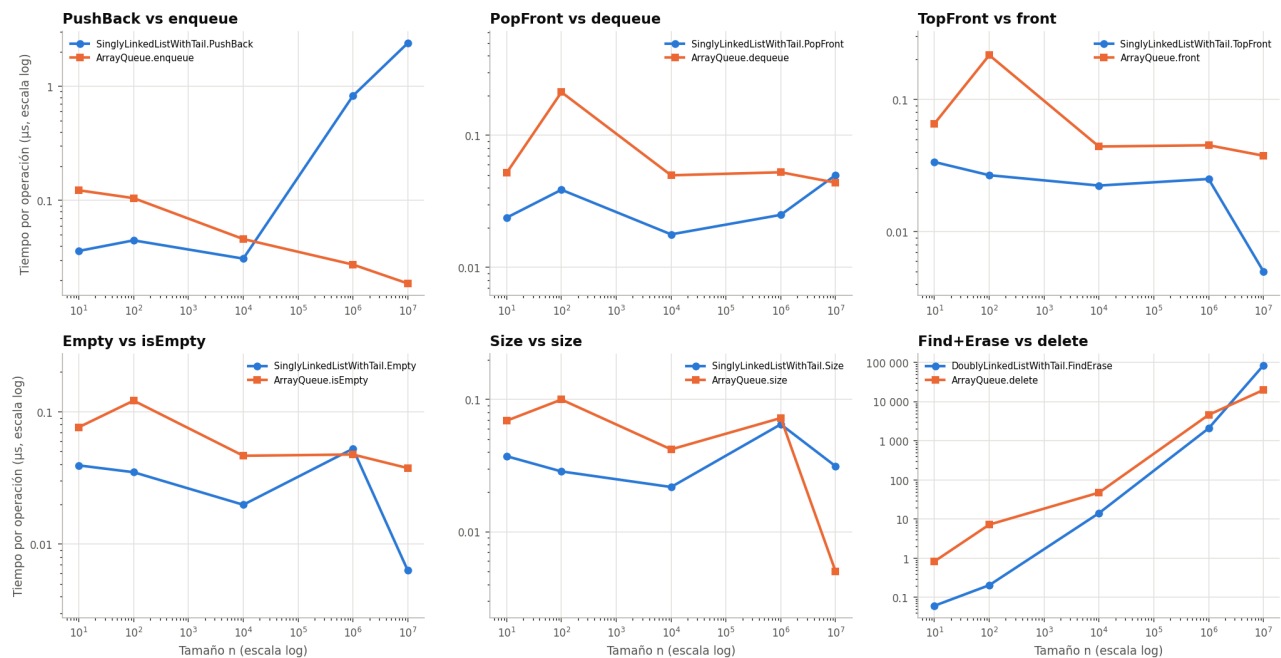


Figura 9. List (simple con cola; doble con cola para Find+Erase) vs. ArrayQueue.

H5 se confirma en lo esencial. En los pares $O(1)$, lista y arreglo quedan en el mismo orden de magnitud y ninguno crece con n . Con $n \leq 10^4$ las operaciones de ArrayStack salen algo más lentas (0.1–0.25 μ s frente a $\approx 0.03 \mu$ s de la lista), pero la diferencia es de uno o dos tics del reloj de 100 ns y desaparece con n grande: con $n = 10^7$, PushFront de la lista tarda 0.062 μ s y push 0.019 μ s; PopFront 0.050 μ s y dequeue 0.044 μ s. La excepción es PushBack de la simple con cola frente a enqueue: con $n = 10^7$ la lista tarda 2.4 μ s y enqueue 0.019 μ s, por el efecto de memoria de la sección 3.5. En delete, ambos crecen linealmente, pero el arreglo es más rápido: con $n = 10^7$, Find+Erase tarda 83 937 μ s frente a 20 808 μ s de la pila (la lista es 4.0 veces más lenta), y 84 579 μ s frente a 19 928 μ s de la cola (4.2 veces). El arreglo paga el desplazamiento de los elementos, pero recorre memoria contigua, que el procesador lee muy rápido; la lista no desplaza nada, pero salta entre nodos

dispersos en memoria, y cada salto puede ser un fallo de caché.

3.5 Comparación con la teoría y diferencias observadas

En conjunto los resultados confirman la complejidad teórica: todos los métodos $O(n)$ crecen en proporción a n y todos los $O(1)$ se mantienen en decenas o pocos cientos de nanosegundos sin crecer con n . Las diferencias con el modelo ideal, que supone que cada operación elemental cuesta lo mismo, se explican así:

- **Resolución del reloj.** En Windows, `System.nanoTime()` avanza en saltos de 100 ns. Una operación $O(1)$ dura menos que eso, así que cada muestra individual mide 0 o 100 ns según caiga o no un salto del reloj durante la llamada. Al promediar cientos o miles de repeticiones se obtiene una estimación del tiempo real (por eso aparecen valores como 0.020 μ s), pero con ruido de ± 1 tic: las diferencias de pocas decenas de nanosegundos entre métodos $O(1)$, o entre tamaños, no son significativas, y un valor de 0.000 μ s significa solo “por debajo de la resolución”. En $n = 10$ y 100 los $O(n)$ apenas se distinguen de los $O(1)$ por la misma razón. Aun así, `.nanoTime()` es la mejor opción: en milisegundos (`Instant/Duration`) todas estas mediciones darían 0.
- **Jerarquía de memoria (caché).** Big-O supone que leer cualquier nodo cuesta lo mismo, pero un dato en caché se lee en ≈ 1 ns y uno en RAM en ≈ 100 ns. Algunos $O(1)$ (`Erase`, `AddBefore` y `AddAfter` en las dobles, `AddAfter` y `PushBack` en las simples) suben con n grande porque, sin cronometrar, antes o después de cada medición se recorre la lista (para buscar el nodo, o para deshacer `PushBack` con un `PopBack` $O(n)$), y eso saca de la caché lo que la operación necesita. Aun así su crecimiento es mucho menor que el de n : `Erase` en la doble con cola crece $\times 60$ entre 10^4 y 10^7 , mientras n crece $\times 1\,000$.
- **Find y los recorridos crecen algo más que n en tamaños grandes.** Entre 10^6 y 10^7 , `Find` en la simple sin cola crece $\times 43$ (n crece $\times 10$). Con listas grandes los nodos ya no caben en la caché, y cada nodo apunta a un `Integer` propio (autoboxing) ubicado en otra zona de memoria, porque los valores están barajados. Cada paso del recorrido cuesta más, aunque la complejidad siga siendo $O(n)$.
- **Peor caso del redimensionamiento.** Duplicar un arreglo grande implica reservar memoria nueva, lo que puede activar el recolector de basura; por eso el peor caso puede crecer algo más que n . El costo amortizado no se ve afectado porque esas duplicaciones son muy raras.
- **Pausas del recolector de basura y del sistema operativo.** Aparecen como muestras aisladas mucho más lentas. La media recortada las descarta; el CSV conserva también la media simple y el máximo.

4. Conclusiones

4.1 Cuándo usar cada estructura

- **Lista simple sin cola:** la mejor cuando solo se opera por el frente (uso tipo pila: `PushFront/PopFront/TopFront`). Es la más simple y la que menos memoria usa por nodo. Debe evitarse si se inserta, consulta o borra por el final, porque esas operaciones son $O(n)$.
- **Lista simple con cola:** la indicada para un patrón FIFO (insertar al final y sacar del frente), con `PushBack` $O(1)$ y sin el costo de prev. No sirve si se necesita `PopBack`, que sigue siendo $O(n)$, ni si se borran nodos en medio.
- **Lista doble sin cola:** útil si se borran o insertan nodos en posiciones arbitrarias (`Erase/AddBefore` $O(1)$ dado el nodo) pero nunca se trabaja por el final. En la práctica rara vez compensa frente a la doble con cola, que cuesta solo un puntero más en toda la lista.
- **Lista doble con cola:** la más completa: todo es $O(1)$ salvo `Find`. Es la elección cuando se opera por ambos extremos y en posiciones intermedias (un deque, una lista de reproducción, una caché

LRU con acceso directo a sus nodos). A cambio usa un puntero prev por nodo y un tail.

- **ArrayStack y ArrayQueue:** para uso LIFO/FIFO puro dan $O(1)$ (amortizado en la inserción) con tiempos del mismo orden que los de la mejor lista, un $\text{delete}(n)$ más rápido que buscar y borrar en una lista, menos memoria por elemento (una referencia en vez de un nodo) y memoria contigua. Sus costos: el resize ocasional $O(n)$, que importa si una operación individual no puede tener picos (sistemas de tiempo real), y $\text{delete}(n)$, que siempre desplaza elementos.

4.2 Pilas y colas en aplicaciones reales

Las **pilas** aparecen donde lo último que llega es lo primero que se atiende: la pila de llamadas de un programa (cada invocación empuja su marco y el return lo saca), deshacer/rehacer en editores, el botón “atrás” del navegador, la validación de paréntesis balanceados, la evaluación de expresiones en notación postfija y el backtracking (DFS, resolver laberintos o sudokus). Las **colas** modelan la atención por orden de llegada: la planificación de procesos del sistema operativo, las colas de impresión, los búferes de teclado y de red, las solicitudes de un servidor web, los sistemas de mensajería entre servicios y el recorrido en anchura (BFS) de grafos y árboles, que exige visitar los nodos en el orden en que se descubren. En casi todos esos casos se implementan sobre un arreglo circular, como aquí, por su localidad de memoria y su bajo costo por elemento.

4.3 Arreglos dinámicos vs. listas enlazadas en Java

	Arreglo dinámico (circular)	Lista enlazada
Ventajas	Acceso por índice $O(1)$. Memoria contigua: aprovecha la caché y la precarga del procesador. Una referencia por elemento. Recorrerlo es muy rápido.	Inserción y borrado $O(1)$ dado el nodo (en la doble), sin desplazar nada. Nunca hay resize : cada inserción cuesta lo mismo. Crece nodo a nodo sin reservar de más.
Desventajas	Insertar o borrar en medio desplaza elementos ($O(n)$). El resize es $O(n)$ en el peor caso ($O(1)$ amortizado). Puede haber hasta la mitad de la capacidad sin usar.	Sin acceso por índice (hay que recorrer). Un objeto nodo por elemento (16–24 bytes extra en la JVM). Nodos dispersos en el heap: cada salto puede ser un fallo de caché.

La lección práctica de este laboratorio es que Big-O dice cómo crece el costo, pero no cuánto vale cada paso. Operaciones con la misma complejidad pueden diferir por la localidad de memoria, el autoboxing de Integer, el JIT o el recolector de basura, y operaciones $O(1)$ pueden encarecerse cuando sus datos ya no están en caché. En Java, para pilas y colas, el arreglo circular es la opción por defecto razonable. La lista doblemente enlazada se justifica cuando hace falta borrar o insertar en medio conservando referencias a los nodos.

5. Repositorio de código

Repositorio GitHub: <https://github.com/USUARIO/lab-estructuras-datos>

Contiene el código fuente (src/main/java), los resultados medidos (results/tiempos*.csv y results/entorno*.txt), el script de graficación (results/graficar.py), las gráficas (results/graficas/) y un README con las instrucciones para compilar y ejecutar.